

Mutation Testing in Practice: Insights from Open-Source Software Developers

Ana B. Sánchez, José A. Parejo, Sergio Segura, Amador Durán, Mike Papadakis

Abstract—Mutation testing drives the creation and improvement of test cases by evaluating their ability to identify synthetic faults. Over the past decades, the technique has gained popularity in academic circles. In practice, however, little is known about its adoption and use. While there are some pilot studies applying mutation testing in industry, the overall usage of mutation testing among developers remains largely unexplored. To fill this gap, this paper presents the results of a qualitative study among open-source developers on the use of mutation testing. Specifically, we report the results of a survey of 104 contributors to open-source projects using a variety of mutation testing tools. The findings of our study provide helpful insights into the use of mutation testing in practice, including its main benefits and limitations. Overall, we observe a high degree of satisfaction with mutation testing across different programming languages and mutation testing tools. Developers find the technique helpful for improving the quality of test suites, detecting bugs, and improving code maintainability. Popularity, usability, and configurability emerge as key factors for the adoption of mutation tools, whereas performance stands overwhelmingly as their main limitation. These results lay the groundwork for new research contributions and tools that meet the needs of developers and boost the widespread adoption of mutation testing.

Index Terms—Mutation Testing, Mutation Tools, Qualitative Study, GitHub

1 INTRODUCTION

MUTATION testing assesses the adequacy of tests based on their ability to detect artificial faults. Such faults are generated by applying *mutation operators*, which introduce simple syntactic changes into the source code of a computer program. For instance, a mutation operator can introduce a fault by changing a relational operator or negating a conditional expression. The injected faults are known as *mutations*, and the new faulty versions of the program under test are known as *mutants*. The adequacy of test suites is measured by the *mutation score* (a.k.a. *mutation coverage*), which is the ratio of detected (a.k.a. *killed*) mutants over the total number of mutants. Mutants that are not killed by the employed test cases, i.e., that remain *alive*, are considered valuable as they showcase ways for improving the test suites.

The origin of mutation testing can be traced back to the 70s in the seminal papers by DeMillo et al. [1, 2]. Since then, mutation testing has flourished until becoming a well-established testing technique and a recurrent research topic in software testing and software engineering venues [3]. Its ability to evaluate and compare the effectiveness of testing approaches has also made mutation testing a popular technique for research purposes. Recent literature reviews on the topic have identified more than 400 research papers on mutation testing and about 130 mutation testing tools—most of them research prototypes—for a variety of programming languages and artefacts, including Java, C,

C++, C#, JavaScript, HTML/CSS, Ruby, and UML models, among others [3, 4].

The impact and widespread adoption of mutation testing in research comes in contrast to the general belief of the current practice. This can be attributed to the lack of evidence of the use of mutation testing in practice. While some case studies [5] and technical reports on large companies such as Google [6] and Facebook [7] have been published, the general use of the mutation technology by practitioners remains unknown. As a result, the overall impact of mutation testing beyond research is still largely unexplored [4, 8]. To address this issue, Sánchez et al. [4] made a quantitative study on the use of mutation testing tools in practice using repository mining. Specifically, they systematically searched and analysed thousands of GitHub repositories that include evidence of the use of mutation testing tools. Among other findings, their results revealed a significant adoption of mutation testing for development purposes in open-source projects, with tools like Infection [9] (PHP) and PIT [10] (Java) being among the most widely used ones.

In this work, we seek to investigate “how” developers are using mutation testing and to gain insights into the benefits and limitations of the method as perceived by the practitioners working on open-source projects. That is, we focus on the usage of mutation testing within software development, including the specific tasks performed with the help of mutation, the perception of the developers on the value provided by the technique, and the key benefits and limitations of existing tools. All-in-all we investigate the extend to which the practitioners views differ from that of researchers with the aim to identify potential points for improvement.

To this end, we aim to answer the following research questions:

- A. B. Sánchez, S. Segura, J. A. Parejo, and A. Durán are with the Universidad de Sevilla, Spain. E-mail: anabsanchez@us.es, sergiosegura@us.es, japarejo@us.es, amador@us.es
- M. Papadakis is with the University of Luxembourg, Luxembourg. E-mail: michail.papadakis@uni.lu

Versión: January 29, 2024

RQ₁ *How mutation testing tools are used in real-world cases?* We aim to study when and how mutation testing tools are used in practice, including the factors that drive their adoption.

RQ₂ *What is the perceived impact of mutation testing in practice?* We aim to study how developers assess the impact of mutation testing in their projects, including both positive (e.g., improving test case quality) and negative points (e.g., execution cost).

RQ₃ *What are the key benefits and limitations of mutation testing tools (as perceived by practitioners)?* We plan to investigate the degree of satisfaction of developers with current tool support and their views on how to improve it.

To answer these questions, we perform a survey on the use of mutation testing among developers of open-source software projects. To do so, we used the data collected by Sánchez et al. [4], based on the analysis of over 3.5K GitHub repositories including evidence of the use of mutation testing tools. That data, allowed us to find the survey participants, i.e., open-source project contributors who are using mutation testing tools. Overall, we collected and reported results based on 104 valid responses from mutation testing practitioners.

The results of our study reveal a high level of satisfaction with mutation testing across various programming languages and mutation testing tools. Developers find the technique beneficial for enhancing their test suites, identifying bugs, and deciding when to stop testing. Additionally, mutation coverage is often used to guide code refactoring (e.g., removing redundant or irrelevant code), contributing to the overall quality of the code. Mutation testing tools are commonly integrated into continuous integration/deployment workflows and build tools. The adoption of mutation tools is mostly driven by their popularity, usability, and configurability. Performance stands out as the primary impediment to the practical implementation of the technique, along with the challenge of seamlessly integrating the tools with Integrated Development Environments (IDEs). Together, these results provide a new perspective on the state of practice of mutation testing, laying the foundation for novel research contributions and tools that meet the needs of developers and foster its broad adoption.

The rest of the article is organised as follows. Section 2 describes the background and related work. Our research methodology is detailed in Section 3. The findings of our study are presented in Section 4. The potential threats to validity are discussed in Section 5. Finally, we present the conclusions of our work in Section 7.

2 BACKGROUND AND RELATED WORK

In this section, we first introduce mutation testing and then discuss related empirical studies on mutation testing, as well as related qualitative studies on software testing.

2.1 Mutation Testing

Mutation testing is a well-known fault-based technique that aims at evaluating and improving the fault-revealing potential of test suites. This technique not only encourages

testers to exercise as much code as possible, but also to uncover possible programming mistakes. The technique relies on injecting faults, known as *mutations*, which are simple syntactic modifications of the code. The faulty versions of the program under test where mutations are injected are known as *mutants*, which, when not detected by test cases, have the potential to reveal real faults Chekam et al. [11].

Two underlying hypotheses support the mutation testing approach. The first one is the so-called *Competent Programmer Hypothesis* [12], which states that programmers tend to write programs that are largely correct, in some sense missing simple cases, similar to mutants in mutation testing. The second one is the so-called *Coupling Effect Hypothesis* [1], which states that tests detecting simple mutants are also able to detect others, including the majority of the more complex ones.

Every mutant as well as the original program are executed against the test suite. The program outputs of the original and mutant programs are compared in order to identify any difference that would indicate that the injected fault was triggered by the test suite and would classify that mutant as detected or *killed*. On the contrary, when the output is the same, the mutant remains *alive* and requires further analysis, as it can point out a deficiency in the fault detection ability of the test suite. This is not always the case, however, because a mutant can turn out to be functionally equivalent to the original program; these are the so-called *semantically-equivalent* mutants. It follows that a tester should aspire to kill as many mutants as possible to increase the detection power of the test suite. The number of killed mutants over the whole set of non-equivalent mutants is called *mutation score* or *mutation coverage*.

The injection of mutations is generally systematised with the development of *mutation tools*, which implement different *mutation operators*. These operators are applied each time a pattern is found in the program (e.g., each appearance of the relational operator ' $>$ ' is replaced by ' $<$ '). As we show in followup sections, more than one hundred mutation testing tools have already been developed and these cover most of the widely-used programming languages, including Java, C/C++, Python or C#. The list of the mutation testing tools has grown significantly in the last years with many mutation testing tools specialised in emerging domains such as Deep Learning [13] and Smart Contracts [14].

2.2 Studies on Mutation Testing

Mutation testing has been extensively analysed and studied in the scientific literature. Several systematic literature reviews [3, 15, 16] have addressed mutation testing from a general point of view, while others have focused on more particular issues within the topic, such as the techniques to reduce its cost [17, 18]. Some other studies have examined various mutation testing tools and compared them following a variety of perspectives. A recent survey by Kintis et al. [19] assesses the effectiveness of three Java mutation testing tools (PIT [10], MuJava [20] and Major [21]) in the detection of faults. The conducted experiments revealed that, while none of these mutation tools completely subsumed the others, an improved version of PIT with research purposes was the most effective tool at inducing test cases that could reveal real faults.

Another group of studies deepens on technical aspects regarding the usage of mutation testing tools, such as efficiency, controllability, compatibility, and integration with a test environment. Although most of these studies focus on Java tools [22, 23], some other programming languages have been considered too, like C# [24]. The study by Delahaye and du Bousquet [22] identifies three different profiles that can influence the election of a mutation tool: teaching, research, and industry. They conclude that PIT is a good choice for the industry and the teaching profiles, where tools should be easy to apply and, in the particular case of the industry, should have a good balance between efficiency and meaningfulness of results.

Regarding the application of mutation tools in the development of software projects, some studies have shown the possible benefits of transferring mutation testing concepts from academia to industry, carrying out empirical studies with open-source projects [11, 25] and industrial projects [5]. In fact, a recent study by Petrović et al. [26] showed that mutation testing has positive long-term effects on the testing practices of developers. Also, some recent studies analyse the use of the technique in large companies. For example, in [6], it is reported how Google implements its own mutation system for seven programming languages and applies a diff-based probabilistic approach to reduce the high computational cost of traditional mutation analysis. The study by Beller et al. [7] reports that more than half of the mutants generated—based on some error-inducing patterns—were not detected by the tests developed at Facebook.

Sánchez et al. [4] reported the findings of a study that investigated the use of mutation testing in practice by looking into GitHub. Specifically, the authors performed a systematic search for GitHub repositories including traces of the use of 127 mutation testing tools. Then, the authors focused on the top ten more widely used tools and manually revised the repositories importing them, over 3.5K. Among others, the results of the study showed 1) a notable upturn in interest in mutation testing in recent years, mostly focused on a small set of highly popular tools; 2) the predominant use of mutation testing in development, followed by teaching & learning, and research; and, 3) some of the most popular mutation testing tools in GitHub are rarely referenced in research papers, e.g., such as Infection [9] or Humbug [27] for PHP.

2.3 Qualitative Studies on Software Testing

Prado and Vincenzi [28] conducted a qualitative study of professionals with unit testing experience aiming to understand how to improve the cognitive support provided by the testing tools, taking into account the perspective of practitioners on their unit testing review practices. The responses of practitioners revealed some primary tasks which require cognitive support, including monitoring of pending and executed unit testing tasks, and navigating across unit testing related artefacts.

Beller et al. [29] reported the results of a large-scale field study with 416 software engineers whose development activity was closely monitored over the course of five months, with the objective of knowing when, how and why developers (do not) test in their IDEs.

A recent qualitative study by Habchi et al. [30] explores the sources, impacts, and mitigation strategies of *flaky* tests through interviews with 14 practitioners. Flaky tests are tests that are non-deterministic, i.e., for the same versions of code and test, can pass and fail on different runs. Their results shows that flakiness stems from interactions between system components, testing infrastructure, and external factors. They also highlight the impact of flakiness on testing practices and product quality and show that the adoption of guidelines together with a stable infrastructure are key measures in mitigating the problem.

In [31], Perry et al. conduct a large-scale user study with 47 participants examining how users interact with an artificial intelligence (AI) code assistant to solve a variety of security tasks. AI code assistants enable developers to write code faster by suggesting code and helping during edition. The authors found that participants with access to an AI code assistant often produced more security vulnerabilities than those without access, possibly due to a false sense that they write more secure code with this kind of assistants.

3 RESEARCH METHODOLOGY

In this section, we describe the research process followed, summarised in Figure 1. Specifically, we provide details about the selection of potential participants, the design of the survey, the protocol for contacting participants, and the qualitative analysis of the collected data. To ensure clarity, Table 1 displays the number of repositories and potential participants associated with each step of the process for each mutation tool.

3.1 Participant Selection

Our goal was to select practitioners who possess experience in working with mutation testing. To accomplish this, we initially identified open-source repositories that included evidence of utilising mutation tools, and then we reached out to the contributors of these repositories and encouraged them to participate in the survey. This process was performed in two primary steps, described below.

3.1.1 Repository Selection (step 1)

We started by identifying open-source projects including evidence of the use of mutation testing tools for development purposes as follows.

Previous repository selection (step 1.1). In the previous study conducted by Sánchez et al. [4], the ten most frequently utilised mutation tools on GitHub (listed in the first column of Table 1) were identified. Following that, they performed a manual examination and categorisation of 3,581 active GitHub repositories that imported these tools. These repositories serve as the foundation for the present study. The number of selected repositories per mutation testing tool is documented in the second column of Table 1.

Development repository filtering (step 1.2). We were interested in surveying developers using mutation testing for development purposes (i.e., as a method to evaluate and improve the quality of test suites) rather than for research

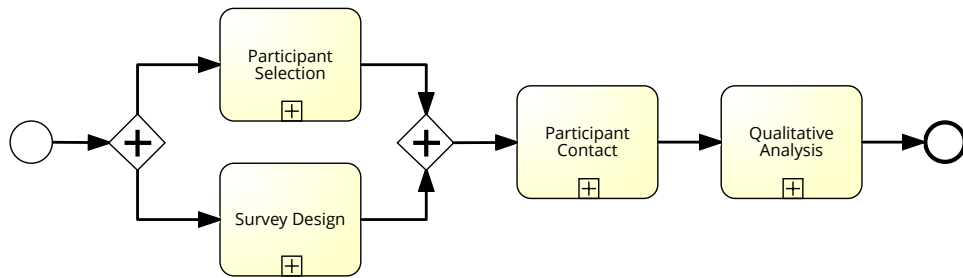


Figure 1. Process followed in the research methodology

Table 1
Evolution of the number of repositories and contributors per tool resulting from each step of our participant selection process

Tools & Refs	Number of Repositories				Number of Contributors		
	Step 1.1	Step 1.2	Step 2.1	Step 2.2	Step 2.2	Step 2.3 (1)	Step 2.3 (2)
Humbug [27]	636	272	242	100	224	222	51
Infection [9]	1,213	704	639	100	700	670	287
Major [21]	10	0	0	0	-	-	-
MuJava [32] [20]	9	0	0	0	-	-	-
Mutant [33]	296	53	51	51	308	285	122
Mutmut [34]	54	20	19	19	61	52	24
MutPy [35] [36]	24	16	15	15	89	88	35
PIT [37] [10]	816	330	325	100	1,295	1,270	419
StrykerJS [38]	487	161	159	100	594	530	218
Stryker.NET [39]	36	18	18	18	104	101	36
Total	3,581	1,574	1,468	503	3,375	3,207	1,192

or teaching. To achieve this, we used the data collected by the manual revision of the repositories performed by Sánchez et al. [4]. In particular, we included repositories classified as “development purposes” and with owners from industry or public institutions only, and excluded those related to teaching (e.g., course materials) and research (e.g., replication packages). This reduced the number of target repositories from 3,581 to 1,574 (44%), leaving two of the tools out of the study, *MuJava* and *Major*, as shown in the third column of Table 1.

3.1.2 Contributor Selection (step 2)

Potential participants were selected among the owners and contributors of the 1,574 repositories previously selected in the three steps described below.

Contributor data extraction (step 2.1). We proceeded to gather information on the owners and contributors of each selected repository with the aim of establishing communication with them. Specifically, we used the REST API¹ of GitHub to automatically extract owner and contributors’ data from the 1,574 selected repositories. In particular, we collected the name, bio, email, Twitter profile, company, location and type (user, organization, or bot) of each contributor/owner. Repositories that were no longer available on GitHub at the date of the data revision (Oct 10th, 2022) were discarded. This reduced the number of target repositories from 1,574 to 1,468, as shown in the fourth column of Table 1. Retrieved data were saved into CSV files for further analysis and to enable replicability.

Repository and contributor filtering (step 2.2). To complete this work with reasonable resources, we focused on the most popular repositories and their most active contributors. Specifically, we considered up to 100 repositories for each tool (ordered by popularity, i.e., number of stars) and up to 100 contributors for each repository (ordered by their number of commits to the project). This made a total of 503 repositories under study (as shown in the fifth column of Table 1) with a potential number of participants of 3,375 based on the number of contributors found on them (as can be seen in the sixth column of Table 1).

Exclusion of bots & contributors without contact information (step 2.3). Finally, we first discarded the contributors who were bots, and secondly, those who did not have email to contact them. Table 1 summarises the resulting data in the seventh and eighth columns, i.e., 3,207 and 1,192 contributors, respectively. This made a total of 503 repositories and 1,192 potential participants for our study.

3.2 Survey Design

To design the survey, we followed Creswell’s guidelines [40] for conducting qualitative research questions. The process took several iterations where four of the authors independently reviewed the questionnaire and discussed it until a consensus was reached (e.g., grouping similar questions together to make them more coherent and concise). The final version of our questionnaire, included in the Appendix, contains 21 questions, 17 closed-ended, in which additional comments can also be included, and 4 open-ended.

The first four questions are related to the participant profile demographics: age, gender and years of experience as a

1. <https://docs.github.com/en/rest>

Table 2
Summarized Survey Core Questions

RQ	Q#	Question	Type	#Resp
RQ ₁	Q ₁	What has been your role on the use of mutation testing in the project?	MC	104
RQ ₁	Q ₂	I use mutation testing. . .	MC	104
RQ ₁	Q ₃	When (and how often) do you compute mutation testing coverage in the projects?	MC	104
RQ ₁	Q ₄	Do you restrict the calculation of mutation coverage results to a specific type of test cases?	MC	103
RQ ₁	Q ₅	How do you invoke the mutation testing tool in the project?	MC	104
RQ ₁	Q ₆	Have you modified the configuration of the mutation testing tool?	OC	104
RQ ₁	Q ₇	Did you participate in the selection of the mutation testing used?	MC	99
RQ ₂	Q ₈	Have mutation testing contributed to improving the quality of the software projects?	MC	104
RQ ₂	Q ₉	Have mutation testing helped in adding new testing practices to the project?	MC	104
RQ ₂	Q ₁₀	Overall, how satisfied are you with the impact of mutation testing in the project?	OC	104
RQ ₃	Q ₁₁	How would you rate the mutation testing tool used in the project regarding the following attributes: usability, performance and useful?	OC	104
RQ ₃	Q ₁₂	What are the main benefits of using the mutation testing tool in this project?	FT	74
RQ ₃	Q ₁₃	What are the main limitations of using the mutation testing tool in this project?	FT	69
RQ ₃	Q ₁₄	What is the single improvement that would significantly improve the mutation testing tool?	FT	57

Legend: RQ: Research Question to which it relates; MC: Multiple Choice; OC: One Choice; FT: Free text
#Resp: number of respondents answering the corresponding question

developer and as a user of mutation testing. The next two questions ask the participants to indicate which mutation tools they have used and how they have learned about them. The rest of the survey contains the core questions aimed to respond the research questions of our study, which are summarised in Table 2. Some of them use a 5-level Likert scale. We include a last question for additional comments or suggestions. Each row in Table 2 shows a question including the research question to which it is related, an identifier, the type of question (MC: multiple choice, OC: one choice, or FT: free text), and the number of respondents that answered it.

3.3 Participant Contact

With the aim of maximising the number of responses, we followed a series of recommendations to improve the survey participation rate [41, 42]: i) *Use the word questionnaire (not survey)*. The word “survey” often has a negative connotation (especially in inboxes). ii) *Ask participants for help*. iii) *Avoid the spam filter*. Avoid unknown senders, such as info@ or noreply@, instead, use a corporate email. iv) *Personalize*. Personalize the subject line and/or, at least, the content of the email, e.g., the greeting. v) *The subject line must be appealing*. vi) *Explain the purpose of the survey*. vii) *Publish the results*. viii) *Be grateful*.

We contacted the participants in two different ways. Firstly, we wrote a personalized email, following the recommendations mentioned above, to the selected contributors. Secondly, we created a GitHub issue for each selected repository requesting the participation of any contributor with experience using the specific mutation testing tool. The e-mail and issue templates are available in the supplementary material [43].

In total, we automatically sent 1,192 emails, one to each selected contributor, and we posted 503 issues, one for each selected repository. The survey was sent out initially on November 29th 2022. A kind reminder was sent later, on December 15th, only to those participants who had replied indicating that they were out of the office or would reply later, and they had not yet done so. During the review process, reviewers required additional responses to increase the significance of the conclusions drawn in this paper. Thus, we performed an additional sent out of the survey on December 7th 2023 (448 emails and 330 issues), we stopped accepting responses on January 2nd 2024.

We received positive feedback from 26 people who replied to our e-mail or GitHub post expressing congratulations for the initiative. Conversely, 8 developers complained about the creation of an issue in their repository. One of the responses received requested permission to share the survey with other colleagues working with mutation testing tools. Other of the participants shared the link to the survey on Twitter encouraging to participate².

3.4 Qualitative Analysis

In this section, we comment on the survey responses received and the analysis of the collected data.

3.4.1 Survey Responses

The 127 questionnaire responses received correspond to a response rate of 10.65% if we consider only the emails sent (notice that we also posted issues on GitHub). The raw data of the responses provided by all the participants is available in the supplementary material [43].

2. https://twitter.com/stryker_mutator/status/1597880416614363136

Twenty-three responses were excluded from the analysis for several reasons. One participant did not consent to the conditions of the survey and did not submit any further information. Three participants did not acknowledge the use of any mutation testing tool. Other nineteen responses were discarded because their participants were not able to identify the mutation testing tool used in their projects and they did not answer any other question in the survey. The final dataset contains 104 valid responses (see the last column of Table 2 for the detailed number of answers per question).

3.4.2 Qualitative Analysis of Survey Data

For the analysis of the qualitative data obtained through the survey, we used a combination of narrative analysis, different kinds of charts, and correlation computed using the Mathews coefficient [44] to analyse the groups of answers that were usually answered together for multiple option questions.

Specifically, for the analysis of responses using a Likert scale, we use divergent stacked bar charts as recommended in [45]. In order to show the distribution of responses among tools, we use horizontal stacked bar charts, with the specific answers provided in the y-axis and the number of participants in the x-axis, using the hue of the bars to denote the specific tools used by the participants (see Figures 2 and 3 in the next section).

4 FINDINGS

In this section, we present the findings after the analysis of the 104 valid responses to the survey (described in the Appendix). Specifically, we first provide demographic data of the participants and then respond to each of the RQs by analysing the answers to the related questions in the questionnaire (see Table 2).

4.1 Demographics

Table 3 shows the demographic information of the 104 valid participants. Most of them were men (93%), between 26 and 55 years old (89%), with a long experience as developers (73.1% have more than ten years of experience), and have been using mutation testing for several years (only 19% of the participants has less than one year experience using mutation testing).

The most popular tool among participants is Infection [9] with 27.9% of participants using it in their projects. The different flavours of Striker are also quite popular among participants: 17.3% of participants use the Javascript version [38], 8.7% use the .Net version [39]. PIT [10] is used by 26.9% of the participants. MuJava [32], Major [21], Stryker for Scala [46], and a custom tool designed for the Rust programming language were each reported as being used by only one participant in the survey. Interestingly, Stryker for Scala and the custom tool were not initially considered for the purposes of repository and contributor selection. It is plausible that the participants who used these tools learned about the survey through a colleague or the tweet mentioned above. Participant P₁₀₆ stated using the “mocha” testing framework as the mutation tool. Given that the

Table 3
Demographic characteristics of participants (n=63)

Property	Value	Freq. & Percentage
Gender	Male	97 (93.3%)
	Female	1 (0.96%)
	Prefer not to answer	6 (5.8%)
Age	< 25	5 (4.8%)
	26 – 35	45 (43.3%)
	36 – 55	48 (46.2%)
	> 56	2 (1.9%)
	Prefer not to answer	4 (3.8%)
Years of experience as developer	1 – 5 years	8 (7.7%)
	6 – 10 years	20 (19.2%)
	> 10 years	76 (73.1%)
Years of experience using mutation testing	< 1 year	20 (19.2%)
	1 – 5 years	68 (65.4%)
	6 – 10 years	14 (13.5%)
	> 10 years	2 (1.9%)
Mutation testing tool	I do not know	1 (0.96%)
	Infection	29 (27.90%)
	Major	1 (0.96%)
	MuJava	1 (0.96%)
	Mutant	9 (8.70%)
	Mutmut	4 (3.80%)
	PIT	28 (26.90%)
	Stryker.Net	9 (8.70%)
	Stryker4s	1 (0.96%)
	StrykerJS	19 (18.30%)
	Custom tool (Rust)	1 (0.96%)

known method of employing Mocha for mutation testing is through integration with StrikerJS³, we interpreted this as an indirect reference to StrikerJS as the mutation tool.

Out of the 104 remaining participants, 41 (39.4%) learned about mutation testing by recommendation of a friend or colleague, and 35 (33.7%) learned it from a blog or publication, as shown in Figure 2.

4.2 Use of Mutation Testing Tools among Software Developers (RQ₁)

In this section, we respond to RQ₁ by analysing the answers to questions Q₁–Q₇ in the questionnaire.

4.2.1 Roles on the Use of Mutation Testing (Q₁)

Most participants (86.5%) took an active role in the use of mutation testing by actively improving their test cases based on the results of the mutation coverage. A small portion of participants (7.7%), however, indicated that they had just seldom checked the mutation score, without taking further actions. A significant portion of the respondents (73.1%) contributed to the integration and setup of the mutation

3. <https://stryker-mutator.io/docs/stryker-js/mocha-runner>

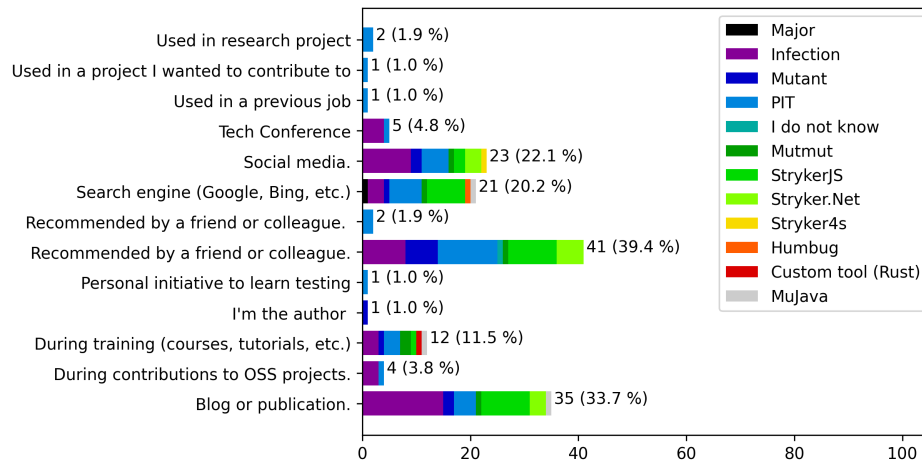


Figure 2. How participants learned about mutation testing

testing tools in their projects, and over half of them (55.8%) changed the configuration of the mutation tool to meet their needs (see Section 4.2.6 for more details).

A positive correlation was found (0.64 using the Matthews correlation coefficient [44]) between participants who contributed to the integration of the mutation testing tools into their projects and those who customised the configuration of the tool.

4.2.2 Aim of Using Mutation Testing (Q_2)

The primary objective of using mutation testing is measuring and improving the quality of the test cases (86.5% of respondents). Among those marking the *others* option (13.5%), some respondents used the free-text answer to specify their motivations for the use of mutation testing—all of them related to software quality improvement. For example, participant P_{30} , user of Infection [9], reported that (s)he uses mutation testing “for removing unnecessary tests and code”. Similarly, P_{17} , user of Mutant [33], uses the technique “to reduce the code to the minimum needed to pass tests”, and P_{29} , also a user of Mutant, reported its use for “reducing unused and unwanted code, arriving at most narrow meaning of the code”. These answers indicate that mutation testing is not only used to improve the fault-detection capability of test cases, but also to improve code efficiency and maintainability through refactoring, such as eliminating redundant or irrelevant code. It is noteworthy that participant P_{103} , user of PIT, indicated the adoption of mutation testing as a measure “to meet the project policies”, implying its institutionalization within the project testing guidelines. However, the participant also noted that they “unfortunately stopped using it on a regular basis after the project terminated”.

4.2.3 When and How Often Mutation Testing Tools are Used (Q_3)

Participants stated that they use mutation testing tools on demand (38.5%), every time test cases are executed (26.9%), before committing changes to the repository (19.2%), and prior to deployment in production (19.2%). Nine participants (8.6%) used the free-text answer to indicate other

options such as “...as often as possible” (participant P_{17} user of Mutant), “...when a pull request is created/modified” (P_{43} user of Infection), and “when I am working on improving tests” (P_{51} user of MutMut). It is noteworthy that a majority of participants (55 out of 104, 52.9%) have integrated the mutation testing tool as part of the build process of their projects (see responses to Q_5 for details).

4.2.4 Specific Type of Test Cases (Q_4)

The majority of participants (70.2%) do not limit the calculation of mutation coverage results to a particular type of test case. However, among those participants who do restrict the calculation, the majority focus on unit tests (73.8%), while a smaller portion considers integration tests (21.4%).

4.2.5 Invocation of the Mutation Testing Tool (Q_5)

A majority of the participants (63.5%) commonly utilise mutation testing tools by integrating them into their continuous integration/deployment workflows using tools like Jenkins or Github Actions. Additionally, build tools such as Maven or Make are employed by 43.3% of the respondents. It is worth noting that approximately two-thirds of the participants also execute mutation tools directly from the command line.

4.2.6 Modification of Configuration (Q_6)

A half of the participants (50%) have modified the configuration of the mutation testing tool used in their projects. This highlights configurability as a relevant issue for the adoption of mutation testing tools. Adjustments include, among others, selecting specific mutation operators, restricting mutation to certain file patterns, or defining timeouts. The most usual adaptations are the filtering of the code to be mutated (34.9% of the reported adaptations), and the selection/customisation of the mutation operators to be applied (25.6% of the reported adaptations). As an example, participant P_{26} reported modifications on Infection for “adjusted mutations, restricted mutated code, adjusted timeouts and limits, specified multiple output/report targets”. Similarly, participant P_{70} , user of Stryker4s, reported modifications for “restricting to file patterns that should be mutated. Restricted the test suites

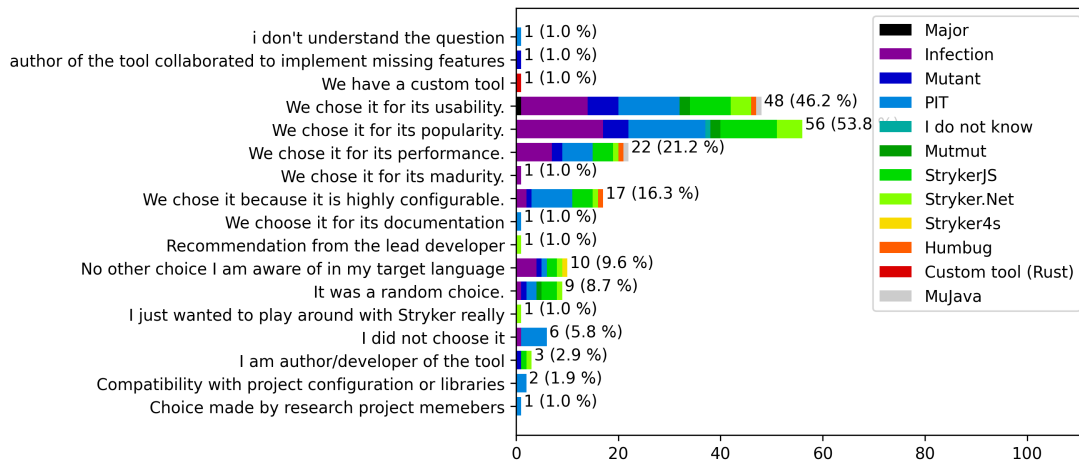


Figure 3. Causes motivating the choice of the mutation testing tools

that the mutation framework should run. Added custom build tasks to run mutation tests check on multiple restricted modules at once with a single build command. Used annotation to escape specific cases, etc.”.

4.2.7 Selection of the Mutation Testing Tool (Q_7)

As summarised in Figure 3, the causes that motivated the choice of the specific mutation testing tool were mainly popularity (53.8%) and usability (46.2%), followed by performance (21.2%) and configurability (16.3%).

Summary of answers to RQ₁

- 1) The majority of participants (86.5%) employ mutation testing tools as a means to assess and enhance the quality of their test cases.
- 2) Some participants (13.5%) go beyond solely focusing on test cases and utilise mutation testing tools to improve the overall quality of their code, e.g., removing redundant or irrelevant code.
- 3) Mutation testing tools are commonly integrated into continuous integration/deployment workflows (63.5%) and build tools (43.3%), although they are also run from the command line (66.7%).
- 4) The choice of a specific mutation testing tool is mainly driven by its popularity (53.8%) and usability (46.2%). A half of the developers (50%) have modified the configuration of the tool to meet their needs.

4.3 Perceived Impact of Mutation Testing in Practice (RQ₂)

In this section, we respond to RQ₂ by examining the answers to questions Q₈–Q₁₀.

4.3.1 Quality Improvement (Q_8)

Overall, a large majority of participants (96.2%) indicated that mutation testing had contributed to improving the quality of their projects. However, we must exercise caution

when interpreting such perceived improvement. As we contacted contributors of repositories using a mutation testing tool—including as a dependency in their project—it is likely that they have a positive opinion about the technique. Those contributors who used a mutation testing tool but did not perceive any positive impact may have removed such tool dependency from their projects and thus were not included in our survey.

When asked about the specific improvements achieved, participants reported that mutation testing helped in improving existing test cases (83.7%), guiding the creation of new test cases (76.0%), detecting more bugs (65.4%), and deciding when to stop testing (48.1%). Figure 4, shows the specific contributions to quality reported by participants.

Some participants utilized the optional free-text answer to elaborate on the ways in which mutation testing proved advantageous for them. For instance, P₂₆ using Infection, stated that “It discovered security issues”. Additionally, several participants such as P₂₉, P₆₈, and P₆₉ leveraged the tool to eliminate dead code, and P₄₈ stated that “Helped prune out unnecessary code branches”.

Four participants (3.8%) reported no improvement in their projects. One participant stated that the tool only provided false positives, while two participants noted that the tool was so slow to run that it was effectively useless or it crashes when they try broadening the scope of its application in the project. These comments highlight potential areas for improvement in mutation testing tools. The fourth participant who perceived no improvement reported that the tool identified missing test coverage, but did not uncover any actual bugs. It is noteworthy, however, that these negative opinions still suggest that mutation testing is providing value, identifying areas lacking coverage and aiding in the development of necessary tests, that would enhance confidence in the software, potentially preventing unnoticed or future bugs.

4.3.2 New Testing Practices (Q_9)

Mutation testing tools appear to have a significant impact on testing methodologies in projects, with 51.9% of participants stating that they contributed to the adoption of new testing practices. When asked about the specific testing practices

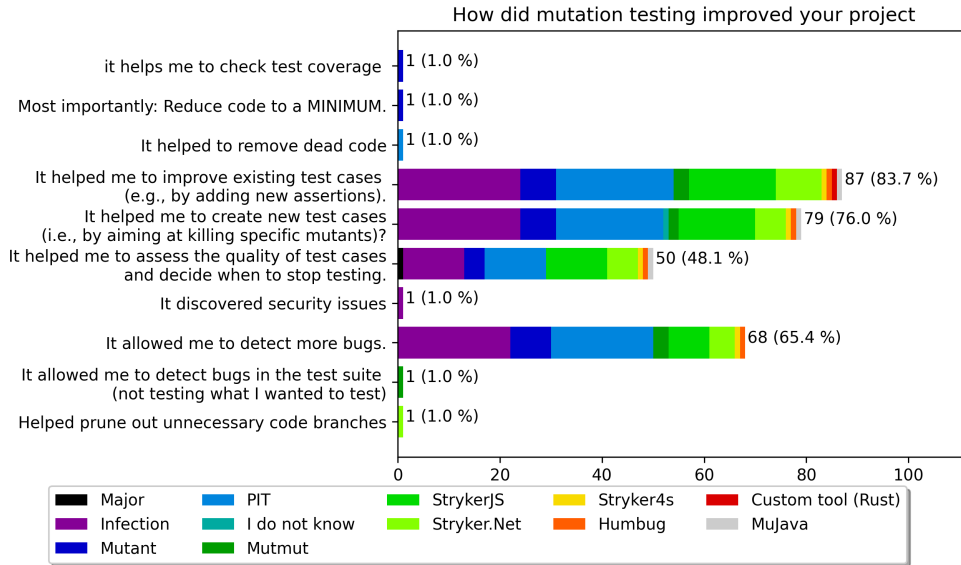


Figure 4. Contributions of mutation testing to quality improvement (multiple responses allowed)

induced, 33.7% of participants reported generating mutation testing coverage reports as part of the build process, 34.6% of participants defined a minimal mutation coverage threshold, and 27.9% enforced a minimal mutation score during the build process. These findings seem to suggest that emerging research on incremental [47] and delta-aware mutation testing [48, 49] is a fruitful research direction. Additionally, in a response to Q_{14} , participant P_{10} , user of MutMut [34], indicated “*mutation testing encourage the team to adopt Test-Driven Development*”, which shows the potential of mutation testing to impact testing practices.

Summary of answers to RQ_2

- 1) The majority of the developers asked (96.2%) consider that mutation testing contributed to improving the quality of their software projects.
- 2) Developers perceive mutation testing as a helpful technique for guiding the improvement (83.7%) and creation (76.0%) of test cases, as well as deciding when to stop testing (48.1%).
- 3) Almost two out of every three participants (65.4%) find mutation testing helpful for detecting bugs in their software
- 4) Mutation testing fosters the adoption of new testing practices, including the integration of mutation coverage computation as a part of the build process (33.7%) and the definition (34.6%) and enforcement (27.9%) of minimal coverage thresholds.
- 5) A vast majority of the participants (89.4%) are either satisfied (49.0%) or very satisfied (40.4%) with the impact of mutation testing in their projects.

4.3.3 General Satisfaction (Q_{10})

The majority of participants (89.4%) are either very satisfied (40.4%) or satisfied (49.0%) with the impact of mutation testing in their project. Nine participants (8.7%) expressed neutrality answering ‘neither satisfied nor dissatisfied’. Only one participant chose the option ‘not satisfied,’ while another single participant selected ‘not at all satisfied’. As previously mentioned, this result must be taken with caution since we contacted contributors of projects where mutation testing is already in use, and therefore it is likely that they are positive about the technique.

4.4 Developer Satisfaction with Mutation Testing Tools (RQ_3)

In this section, we respond to RQ_3 by analysing the answers to questions Q_{11} – Q_{14} .

4.4.1 Usability, Performance, and Usefulness (Q_{11})

Figure 5 shows a diagram of divergent stacked bars summarising the answers to this question. The specific distribution of responses regarding usability, performance, and usefulness for each specific tool is depicted in Figure 6. A minimum of 71% of respondents provided positive responses (agree or strongly agree) regarding satisfaction on all the specific aspects evaluated through the questionnaire: usability, performance, and usefulness. Conversely, negative responses (disagree or strongly disagree) on any aspect

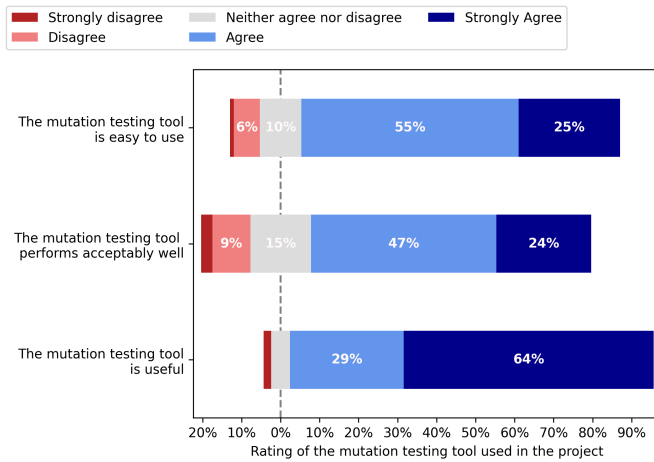


Figure 5. Global perception of the usability, performance, and usefulness of mutation testing tools

evaluated were consistently below 14%. Overall, the mutation testing tools are perceived as useful (93% of positive responses), user-friendly (80% of positive responses), and performant (71% of positive responses). Negative views are mostly related to performance (14% of negative responses), followed by usability (7% of negative responses).

4.4.2 Main Benefits of Mutation Testing Tools (Q_{12})

When asked about the main benefits of using mutation testing tools in their projects—a free-text question—the participants identified as key benefits the enhancement of test suite quality, the improvement of test cases, the identification of missing test cases, and the detection of bugs. As expected, this is in line with the responses to Q_8 , regarding how mutation testing has contributed to improving the quality of their projects.

Figure 7, shows a word cloud generated from the responses, in which the size of each word is determined by the number of appearances in the answers. Common terms such as “mutation”, “test”, “testing”, “code”, and “cases” were excluded from the word cloud as they appeared in almost every response. Notably, the words “quality” and “improving” (and the related term “improve”) appear prominently, indicating that they were frequently mentioned by the participants. Some participants stressed the ability of mutation testing to measure code coverage and identify relevant test inputs: “When I code a feature I have in mind all the corner cases, but then not always write tests for them - then mutation tests will help me to ‘remember’ finding hidden bugs and missing test cases” (P₇₅ using Infection); whereas other emphasise the ability of the technique to improve test oracles: “To prove code is tested, necessary, and that the code and test are making the strongest assertions they can.” (P₇₈ using Mutant). P₁₁₃ using StrykerJS stated that the main benefit was “Helping developers who don’t do test first development”. More generally, some participants pointed out that the main benefit of mutation testing was that developers “feel more secure about the quality of the unit tests” (P₆₃ using PIT), or that it contributes to “finding undiscovered bugs and help increasing test coverage” (P₃₅ using PIT).

4.4.3 Main Limitations of Mutation Testing Tools (Q_{13})

When asked about the limitations of the tools, performance emerged as the most significant issue for the users of 6 out of the 10 mutation testing tool under study. Specifically, 17 out of the 36 answers referred to performance as a key limitation.

This finding is highlighted in the word cloud shown in Figure 8, generated from the answers to this question. The words “performance”, “speed”, and “time” appeared frequently and were particularly prominent in the cloud, indicating that they were major concerns for the participants. For instance, P₂₆ stated that “tools’ performance on large projects makes it only viable for libraries”, and P₃₁ claimed that “it’s impossible to run on the whole project, due to performance/scale”. Participants P₄₆, P₄₇, and P₅₁ just answered “performance”, “performance”, and “speed” respectively. These findings suggest that, despite research advances in cost-reduction techniques for mutation testing, at least those included in the tools used (e.g., bytecode mutation and coverage-driven mutant execution), performance is still a major obstacle for the adoption of mutation testing in practice. Here it must be noted that the recent advances on learning-based mutant selection and mutant prioritisation [50, 51, 49] that promise to reduce overheads, have not been integrated on the studied tools, indicating the need for a better tooling and adoption of research advances.

Other responses pointed the need for improvement of the tools in terms of usability (e.g., “ease of use & setup overhead”) and configurability (“...Conditional tests, depending on specific versions of language or library are not taken into account.”).

4.4.4 Single Improvement for Mutation Testing Tools (Q_{14})

Consistently with the above answers, when developers were asked on what single improvement they would like to see on the mutation testing tools, a majority of participants (25 out of 57) requested better performance. Other responses referred to usability and configurability issues such as “an IDE integration. I would test any class/method in one click” (P₃₇, using Infection [9]), “good documentation on how to set up in different environments (build tools, CI/CD)” (P₆₃, using PIT [10]), and “a better suppression system” (P₆₉, also using PIT [10]). Finally one participant pointed out that the main single improvement would be “Mostly just educating developers around what it is” (P₁₀₄ using Infection).

Summary of answers to RQ₃

- 1) The majority of participants (89.4%) are satisfied with the mutation tool used as a way to enhance test suites, detect bugs and, more generally, improve code quality.
- 2) Seven out of every ten developers are satisfied with the usefulness, usability and performance of the mutation tool used in their projects.
- 3) Performance clearly emerges as the main limitation of mutation tools and as the most pressing concern for developers.

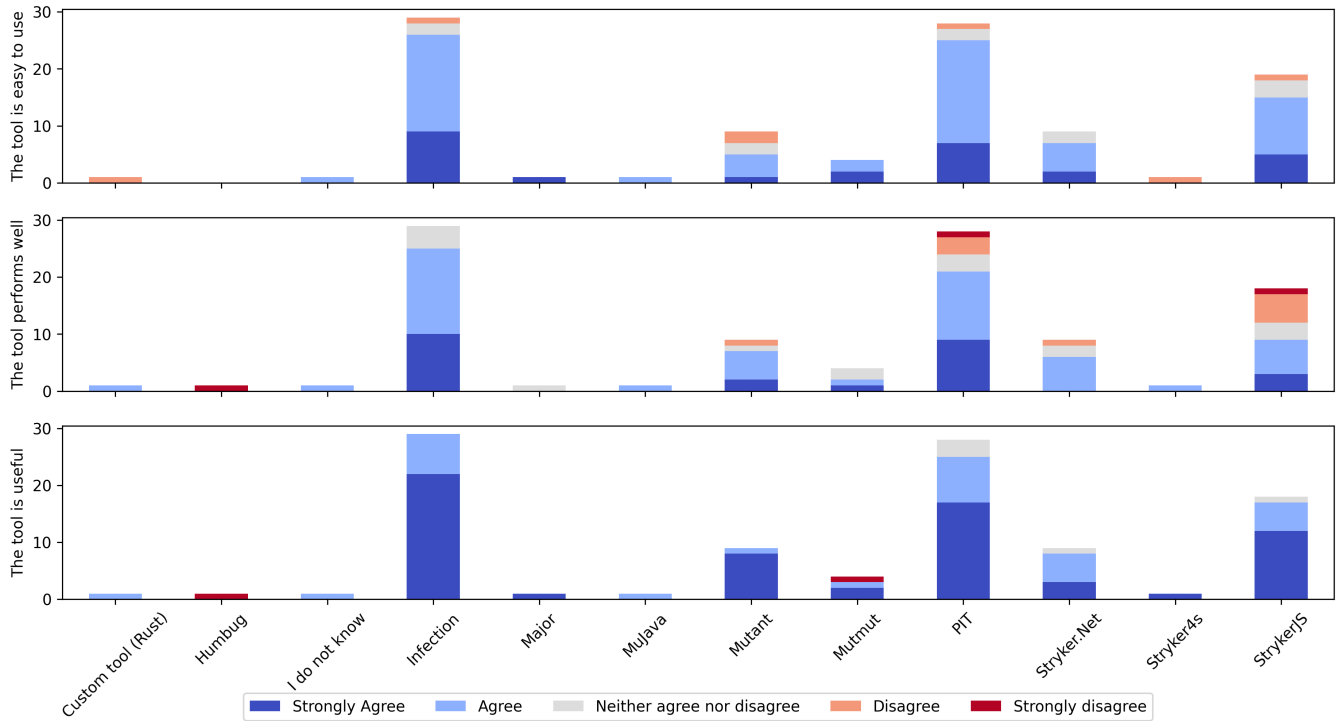


Figure 6. Distribution of responses regarding usability, performance, and usefulness per specific mutation testing tool



Figure 7. Word cloud generated from the responses of participants about the benefits of mutation testing tools

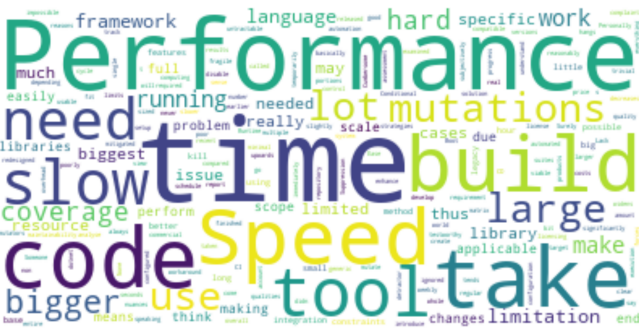


Figure 8. Word cloud generated from the responses of participants about the limitations of mutation testing tools

5 THREATS TO VALIDITY

To facilitate a comprehensive examination of potential validity threats to our work, we use the terminology and taxonomy established in [52]. We have also tailored the threats to survey studies outlined in [53] to align with the categories in this taxonomy. Next, we detail the specific threats to validity for each category and outline the corresponding actions taken to mitigate them.

5.1 Internal validity

Internal validity refers to whether there is sufficient evidence to support the conclusions and the sources of bias that could compromise those conclusions.

Our study draws on the 3,581 GitHub repositories using mutation testing tools resulting from our previous work [4], which is based on the assumption that software projects importing a mutation testing tool, such as a library, most likely use or have used that tool at some point, although this might not be always true. This threat was minimised by the number of manually reviewed repositories, over 3.5K, which dilutes the potential effect of these unlikely cases where the mutation tool is imported, but not used.

In the current study, we only selected the repositories classified as development and with owners at industry or non-academic public institutions. The manual review and classification of the repositories in our previous work could have also threatened the validity of the results, since some repositories could have been misclassified. However, the authors participated actively in the classification process following a common review procedure and discussing the cases of doubts in several working sessions until reaching a consensus.

In this work, we proceeded to collect information about the owner and contributors of the selected GitHub repositories. The participants data collection method could also threaten the validity of our work. To mitigate this threat, we resorted to automated queries on the REST API of GitHub to obtain the data of contributors and owners of target repositories.

5.2 External validity

External validity in surveys focuses largely on the representativeness of the sample for the target population of the study. In this sense, the implications of the repository mining, and tool selection procedures are discussed.

In order to study the use of mutation tools in practice, we focused on the GitHub repositories selected in our previous work [4], as we mentioned in the previous section. Widening the scope of our work to other platforms beyond GitHub (e.g., Bitbucket [54] or GitLab [55]) could have yielded different results. However, the size and the popularity of GitHub in related mining studies make us confident in the validity of the reported trends.

To ensure that our study was feasible with our available resources, we limited the selection of repositories to a maximum of 100 for each tool (ordered by popularity, i.e., number of stars) and a maximum of 100 contributors for each repository (ordered by contributions, i.e., number of commits). This resulted in 503 repositories under study with 1192 potential participants, as shown in Table 1. We are aware that this may have left contributors who have worked with mutation tools in their repositories out of the survey. However, to reduce this possibility, we worked in two directions. Firstly, as we have already mentioned, we contacted the 100 contributors with the highest number of contributions in the repository, in order to reach the most valuable and most active contributors in the project. Secondly, we also posted an issue with the link to the survey on the most popular repositories using the mutation tools of the study, trying to reach any interested and mutation-experienced contributor of the repository. In addition to this, we also allowed contributors to share the survey directly to other colleagues in the project who were working with mutation.

The method followed for selecting repositories introduces bias, as only those repositories using mutation testing tools as libraries or dependencies were considered. Contributors who used such tools but discontinued were not selected. This threat is hardly mitigable, as finding such dependencies or libraries would require going through the entire version history of each repository in GitHub, making the study practically unfeasible. Consequently, the findings and conclusions of our survey are not universally applicable to all open-source developers who used mutation testing. Instead, they specifically pertain to developers who continued using and finding value in the tool. This selection bias may inadvertently overlook significant issues and drawbacks associated with mutation testing. Despite this, however, this study provides valuable insights into how mutation testing is currently used in practice, highlighting the perspectives of developers on key benefits, drawbacks, and potential improvements in mutation testing tools.

Another factor affecting the generalizability of the study's results is the limited number of responses for some mutation testing tools, as indicated in Table 3. To minimize the impact of this limitation on the study's conclusions, we have deliberately refrained from making specific assertions about individual mutation testing tools. The research questions and their corresponding answers are formulated to address mutation testing tools as a whole, rather than focusing on any particular tool.

5.3 Construct validity

Construct validity evaluates whether a measurement tool really represents what we are interested in measuring. In this study, most of the questions in our survey focus on the actual practices of participants regarding mutation testing in their projects and their usage of the libraries and tools. For such types of questions, we provided as default options the practices that according to the existing literature and authors' experience are more common. However, in all such cases, multiple responses were allowed. Also, we provided an open text option for participants to provide their views to avoid a systematic bias towards authors' perceptions. For some questions, Likert scales were used to measure attitudes and opinions with a greater degree of nuance than a simple *yes/no* question. For the design of such questions and scales, we followed the guidelines provided in [56]. Specifically, we formulated the statement of the question, followed by a series of five labelled options, where participants choose the option that best corresponds with how they feel about the statement. The specific labels used in such questions were "Strongly disagree", "Disagree", "Neither Agree nor Disagree", "Agree", and "Strongly Agree".

In order to ensure that we introduce no bias in responses due to the redaction, structure, and type of questions included in the questionnaire, we followed Creswell's guidelines [40] for conducting qualitative research questions.

6 OPEN RESEARCH DIRECTIONS

The results of this work provide a new perspective on the state of practice of mutation testing, laying the foundation for novel research contributions and tools that meet the needs of developers and foster its broad adoption.

During the analysis of the responses provided by participants, the authors realized that some interesting benefits and problems of the application of mutation testing tools reported by developers were provided through the free text options of the questionnaire. This fact, along with difficulties for the design and analysis of the survey suggest that a grounded theory on how mutation testing impacts on software development could be a suitable research direction in the future.

The concerns of developers with the performance of mutation testing tools points out another research direction of practical interest. Using the contact information provided voluntarily by participants who wanted to receive additional information about the results of this work and considering that performance was pointed out as an important issue of mutation testing tools, we traced back a set of 16 different associated open-source repositories for six of the

tools. This listing of repositories has been added to the laboratory package of this work [43] and could be used by mutation tools developers and researchers as suitable experimental subjects where the improvements on the performance of such tools could be applied and validated.

Finally, this paper performed a wide and—to some extent—shallow study of the use in practice of mutation testing by developers of open-source software. However, a deeper understanding of the phenomenon is still required, and in this sense a study performing in-depth interviews with industry practitioners applying mutation testing would be a great complement for this paper, emerging as an open research direction for future work.

7 CONCLUSIONS

This article presents the findings of a qualitative study on the use of mutation testing in practice. To this end, we contacted developers of GitHub repositories using mutation testing tools and invited them to participate in a survey, obtaining a total of 104 valid responses. Results reveal a strong sense of satisfaction among developers regarding mutation testing as a means to enhance test cases, detect bugs, decide when to stop testing, and improve overall code quality. Mutation coverage computation is mostly automated and integrated as a part of continuous integration and deployment workflows. Developers mostly choose mutation testing tools based on their popularity, but usability and configurability emerge as key adoption drivers too. Mutation testing fosters the adoption of new testing practices such as the definition and enforcement of minimal coverage thresholds. Among the points for improvement, performance emerges overwhelmingly as the most pressing concern for developers. This suggests that further efforts are required to bridge the gap between research advancements in cost-reduction techniques for mutation testing and their application in practice.

ACKNOWLEDGMENTS

This work has been partially supported by grants PID2021-126227NB-C21 and PID2021-126227NB-C22 funded by MCIN/AEI/10.13039/501100011033/FEDER, UE, and grants TED2021-131023B-C21 and TED2021-131023B-C22 funded by MCIN/AEI/10.13039/501100011033 and by European Union “NextGenerationEU”/PRTR». Ana B. Sánchez was supported by the VI Plan Propio de Investigación y Transferencia of Universidad de Sevilla 2021 [VI PPIT-US].

REFERENCES

- [1] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Hints on test data selection: Help for the practicing programmer,” *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [2] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Program mutation: A new approach to program testing,” in *Infotech State of the Art Report, Software Testing*, 1979, pp. 107–126.
- [3] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, “Chapter six - mutation testing advances: An analysis and survey,” ser. *Advances in Computers*, A. M. Memon, Ed. Elsevier, 2019, vol. 112, pp. 275 – 378.
- [4] A. B. Sánchez, P. Delgado-Pérez, I. Medina-Bulo, and S. Segura, “Mutation testing in the wild: Findings from github,” *Empirical Software Engineering*, vol. 27, no. 132, 2022.
- [5] P. Delgado-Pérez, I. Habli, S. Gregory, R. Alexander, J. Clark, and I. Medina-Bulo, “Evaluation of mutation testing in a nuclear industry case study,” *IEEE Transactions on Reliability*, vol. 67, no. 4, pp. 1406–1419, 2018.
- [6] G. Petrovic, M. Ivankovic, G. Fraser, and R. Just, “Practical mutation testing at scale: A view from Google,” *IEEE Transactions on Software Engineering*, vol. 48, no. 10, p. 3900–3912, 2022.
- [7] M. Beller, C. Wong, J. Bader, A. Scott, M. Machalica, S. Chandra, and E. Meijer, “What it would take to use mutation testing in industry – a study at Facebook,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2021, pp. 268–277.
- [8] A. Arcuri, “An experience report on applying software testing academic results in industry: we need usable automated test generation,” *Empirical Software Engineering*, vol. 23, no. 4, pp. 1959–1981, 2018.
- [9] “Infection,” <https://infection.github.io/>.
- [10] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, “PIT: A practical mutation testing tool for Java (demo),” in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ser. ISSTA 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 449–452.
- [11] T. T. Chekam, M. Papadakis, Y. L. Traon, and M. Harman, “An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption,” in *Proceedings of the 39th International Conference on Software Engineering, ICSE 2017, Buenos Aires, Argentina, May 20-28, 2017*, S. Uchitel, A. Orso, and M. P. Robillard, Eds. IEEE / ACM, 2017, pp. 597–608. [Online]. Available: <https://doi.org/10.1109/ICSE.2017.61>
- [12] A. T. Acree, T. A. Budd, R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Mutation analysis,” Georgia Institute of Technology, Atlanta, Georgia, techreport GIT-ICS-79/08, 1979.
- [13] L. Ma, F. Zhang, J. Sun, M. Xue, B. Li, F. Juefei-Xu, C. Xie, L. Li, Y. Liu, J. Zhao, and Y. Wang, “Deep-mutation: Mutation testing of deep learning systems,” in *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*, 2018, pp. 100–111.
- [14] Z. Li, H. Wu, J. Xu, X. Wang, L. Zhang, and Z. Chen, “Musc: A tool for mutation testing of Ethereum smart contract,” in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1198–1201.
- [15] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [16] A. J. Offutt and R. H. Untch, *Mutation 2000: Uniting the Orthogonal*. USA: Kluwer Academic Publishers, 2001,

- p. 34–44.
- [17] A. V. Pizzoleto, F. C. Ferrari, J. Offutt, L. Fernandes, and M. Ribeiro, "A systematic literature review of techniques and metrics to reduce the cost of mutation testing," *Journal of Systems and Software*, vol. 157, p. 110388, 2019.
- [18] M. P. Usaola and P. R. Mateo, "Mutation testing cost reduction techniques: A survey," *IEEE Software*, vol. 27, no. 3, pp. 80–86, 2010.
- [19] M. Kintis, M. Papadakis, A. Papadopoulos, E. Valvis, N. Malevris, and Y. Le Traon, "How effective are mutation testing tools? An empirical analysis of java mutation testing tools with manual analysis and real faults," *Empirical Software Engineering*, vol. 23, no. 4, pp. 2426–2463, 2018.
- [20] Y.-S. Ma, J. Offutt, and Y. R. Kwon, "MuJava: an automated class mutation system," *Software Testing, Verification and Reliability*, vol. 15, no. 2, pp. 97–133, 2005.
- [21] R. Just, "The Major mutation framework: Efficient and scalable mutation analysis for Java," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 433–436.
- [22] M. Delahaye and L. du Bousquet, "Selecting a software engineering tool: lessons learnt from mutation analysis," *Software: Practice and Experience*, vol. 45, no. 7, pp. 875–891, 2015.
- [23] A. Márki and B. Lindström, "Mutation tools for Java," in *Proceedings of the Symposium on Applied Computing*, ser. SAC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1364–1415.
- [24] S. Uzunbayir and K. Kurtel, "An analysis on mutation testing tools for C# programming language," in *2019 4th International Conference on Computer Science and Engineering (UBMK)*, 2019, pp. 439–443.
- [25] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 654–665.
- [26] G. Petrović, M. Ivanković, G. Fraser, and R. Just, "Does mutation testing improve testing practices?" in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2021, pp. 910–921.
- [27] "Humbug," <https://github.com/humbug/humbug>.
- [28] M. P. Prado and A. M. R. Vincenzi, "Towards cognitive support for unit testing: A qualitative study with practitioners," *Journal of Systems and Software*, vol. 141, pp. 66–84, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121218300529>
- [29] M. Beller, G. Gousios, A. Panichella, and A. Zaidman, "When, how, and why developers (do not) test in their ides," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 179–190. [Online]. Available: <https://doi.org/10.1145/2786805.2786843>
- [30] S. Habchi, G. Haben, M. Papadakis, M. Cordy, and Y. L. Traon, "A qualitative study on the sources, impacts, and mitigation strategies of flaky tests," in *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 2022, pp. 244–255. [Online]. Available: <https://doi.org/10.1109/ICST53961.2022.00034>
- [31] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with ai assistants?" 2022. [Online]. Available: <https://arxiv.org/abs/2211.03622>
- [32] "MuJava. <https://github.com/jeffoffutt/muJava>," accessed in October 2021.
- [33] "Mutant. <https://github.com/mbj/mutant>," accessed in October 2021.
- [34] "Mutmut. <https://github.com/boxed/mutmut>," accessed in October 2021.
- [35] "MutPy. <https://github.com/mutpy/mutpy>," accessed in October 2021.
- [36] A. Derezińska and K. Hałas, "Analysis of mutation operators for the Python language," in *Proceedings of the Ninth International Conference on Dependability and Complex Systems DepCoS-RELCOMEX*. June 30–July 4, 2014, Brunów, Poland. Springer, 2014, pp. 155–164.
- [37] "PIT. <https://github.com/hcoles/pitest>," accessed in October 2021.
- [38] "StrykerJS," <https://stryker-mutator.io/docs/stryker-js/introduction>.
- [39] "Stryker-NET," <https://stryker-mutator.io/docs/stryker-net/Introduction>.
- [40] J. W. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. California: Fourth Edition SAGE Publications, Inc, 2014.
- [41] W. Fan and Z. Yan, "Factors affecting response rates of the web survey: A systematic review," *Computers in Human Behavior*, vol. 26, no. 2, pp. 132–139, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0747563209001708>
- [42] T. V. Greer, N. Chuchinprakarn, and S. Seshadri, "Likelihood of participating in mail survey research: Business respondents' perspectives," *Industrial Marketing Management*, vol. 29, no. 2, pp. 97–109, 2000. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0019850198000388>
- [43] "Supplementary material of the article [Online]. Available at <https://zenodo.org/record/8032129>."
- [44] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation," *BMC genomics*, vol. 21, pp. 1–13, 2020.
- [45] N. B. Robbins and R. M. Heiberger, "Plotting likert and other rating scales," in *2011 joint statistical meeting*, vol. 1. American Statistical Association, 2011.
- [46] "Stryker4S," <https://stryker-mutator.io/docs/stryker4s/getting-started>.
- [47] M. A. Cachia, M. Micallef, and C. Colombo, "Towards incremental mutation testing," in *Proceedings of the 2013 Validation Strategies for Software Evolution Workshop, VSSE 2013, Rome, Italy, March 16, 2013*, ser. Electronic Notes in Theoretical Computer Science, H. Chockler, Ed., vol. 294. Elsevier, 2013, pp. 2–11. [Online]. Available: <https://doi.org/10.1016/j.entcs.2013.02.012>
- [48] W. Ma, T. Laurent, M. Ojdanic, T. T. Chekam,

- A. Ventresque, and M. Papadakis, "Commit-aware mutation testing," in *IEEE International Conference on Software Maintenance and Evolution, ICSME 2020, Adelaide, Australia, September 28 - October 2, 2020*. IEEE, 2020, pp. 394–405. [Online]. Available: <https://doi.org/10.1109/ICSME46990.2020.00045>
- [49] W. Ma, T. T. Chekam, M. Papadakis, and M. Harman, "Mudelta: Delta-oriented mutation testing at commit time," in *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 2021, pp. 897–909. [Online]. Available: <https://doi.org/10.1109/ICSE43902.2021.00086>
- [50] T. T. Chekam, M. Papadakis, T. F. Bissyandé, Y. L. Traon, and K. Sen, "Selecting fault revealing mutants," *Empir. Softw. Eng.*, vol. 25, no. 1, pp. 434–487, 2020. [Online]. Available: <https://doi.org/10.1007/s10664-019-09778-7>
- [51] A. Garg, M. Ojdanic, R. Degiovanni, T. T. Chekam, M. Papadakis, and Y. L. Traon, "Cerebro: Static subsuming mutant selection," *IEEE Trans. Software Eng.*, vol. 49, no. 1, pp. 24–43, 2023. [Online]. Available: <https://doi.org/10.1109/TSE.2022.3140510>
- [52] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering*. Springer Science & Business Media, 2012.
- [53] M. Ciolkowski, O. Laitenberger, S. Vegas, and S. Biffl, *Practical experiences in the design and conduct of surveys in empirical software engineering*. Springer, 2003.
- [54] "Bitbucket. <https://bitbucket.org/>," accessed in April 2023.
- [55] "GitLab. <https://about.gitlab.com/>," accessed in April 2023.
- [56] A. T. Jebb, V. Ng, and L. Tay, "A review of key likert scale development advances: 1995–2019," *Frontiers in psychology*, vol. 12, p. 637547, 2021.